

Listes et gestion des ennemis simples

Il y a plusieurs méthodes simples pour rendre la gestion des ennemis plus efficace dans un jeu vidéo avec Unity2d

1.1 Utilisation de listes

Une liste est une structure de données qui permet de stocker plusieurs éléments du même type. En utilisant des listes pour gérer les ennemis, on peut facilement ajouter, supprimer et itérer sur les ennemis présents dans le jeu.

1.1.1 Créer une liste

Pour créer une liste, on peut utiliser la classe `List<GameObject>`. Si vous devez stocker des float, des strings ou d'autres types, vous pouvez remplacer `GameObject` par le type souhaité.

Dans l'exemple ci-dessous, nous créons deux listes : une pour la banque d'ennemis (tous les ennemis disponibles) et une pour les ennemis actuellement actifs dans le jeu. Dans l'inspecteur, nous pouvons glisser-déposer des prefabs d'ennemis dans la liste banqueEnnemis. On choisit un ennemi aléatoire dans la banque et on l'instancie à un point de création défini.

On peut faire la même chose avec des bonus, des armes, des pièces de monnaie, etc.

```
using System.Collections.Generic;
using UnityEngine;
public class EnemyManager : MonoBehaviour
{
    public List<GameObject> banqueEnnemis; // Liste pour stocker les ennemis
    public List<GameObject> ennemisEnJeu; // Liste pour stocker les ennemis actifs dans le jeu
    public Transform pointCreation; // Point de création des ennemis

    void Start()
    {
        InvokeRepeating("GenererEnnemi", 0f, 30.0f); // Génère un ennemi toutes les 30 secondes
    }

    void GenererEnnemi()
    {
        // Exemple d'ajout d'un ennemi à la liste
        if(ennemisEnJeu.Count <= 10) // Limite le nombre d'ennemis en jeu
        {
            GameObject ennemiAleatoire = banqueEnnemis[Random.Range(0, banqueEnnemis.Count)];
            GameObject nouvelEnnemi = Instantiate(ennemiAleatoire, pointCreation.position,
            pointCreation.rotation);
            nouvelEnnemi.SetActive(true);
            ennemisEnJeu.Add(nouvelEnnemi); // Ajouter l'ennemi à la liste des ennemis en jeu
        }
    }
}
```

1.1.2 Créer un ennemi qui patrouille une zone

Vous pouvez créer un script simple pour faire patrouiller un ennemi entre deux points. On prépare une variable pour stocker le point cible actuel, et on utilise la fonction `Vector2.MoveTowards` pour déplacer l'ennemi vers ce point. Lorsqu'il atteint le point, on change la cible pour qu'il retourne à l'autre point.

```
using UnityEngine;

public class EnnemiPatrouilleur : MonoBehaviour
{
    public Vector2 pointA; // Premier point de patrouille
    public Vector2 pointB; // Deuxième point de patrouille
    public float vitesse = 2f; // Vitesse de déplacement

    private Vector2 cible; // Point cible actuel
    private SpriteRenderer spriteRenderer;

    void Start()
    {
        cible = pointB; // Commencer par se diriger vers le point B
        spriteRenderer = GetComponent<SpriteRenderer>();
    }

    void Update()
    {
        // Déplacer l'ennemi vers la cible
        transform.position = Vector2.MoveTowards(transform.position, cible, vitesse * Time.deltaTime);

        // Si l'ennemi atteint la cible, changer de cible
        if (Vector2.Distance(transform.position, cible) < 0.1f)
        {
            if (cible == pointB)
            {
                cible = pointA;
                spriteRenderer.flipX = true; // Retourner le sprite
            }
            else
            {
                cible = pointB;
                spriteRenderer.flipX = false; // Retourner le sprite
            }
        }
    }
}
```

1.2 Créer un ennemi qui suit le joueur

Pour faire en sorte qu'un ennemi suive le joueur, on peut utiliser la fonction `Vector2.MoveTowards` pour déplacer l'ennemi vers la position du joueur à chaque frame seulement si le joueur est dans une certaine portée. De plus, on peut ajouter une

animation de déplacement et gérer l'orientation du sprite en fonction de la position du joueur. Si le joueur sort de la portée, l'ennemi retourne à sa position initiale.

```
using UnityEngine;

public class EnnemiSuiveur : MonoBehaviour
{
    public Transform cible; // Référence au transform du joueur
    public Transform positionInitiale; // Position initiale de l'ennemi
    public float vitesse = 3f; // Vitesse de déplacement
    public float porteeDetection = 5f; // Portée de détection du joueur

    private SpriteRenderer spriteRenderer;
    private Animator animator;

    void Start()
    {
        spriteRenderer = GetComponent<SpriteRenderer>();
        animator = GetComponent<Animator>();
        positionInitiale.position = transform.position; // Enregistrer la position initiale
    }

    void Update()
    {
        float distanceJoueur = Vector2.Distance(transform.position, cible.position);
        float distanceInitiale = Vector2.Distance(transform.position, positionInitiale.position);
        // Si le joueur est dans la portée de détection, suivre le joueur
        if (distanceJoueur < porteeDetection)
        {
            transform.position = Vector2.MoveTowards(transform.position, cible.position, vitesse *
Time.deltaTime);
            animator.SetBool("isMoving", true); // Activer l'animation de déplacement
            // Gérer le flip du sprite en fonction de la position du joueur
            if (cible.position.x < transform.position.x)
            {
                spriteRenderer.flipX = true; // Le joueur est à gauche
            }
            else
            {
                spriteRenderer.flipX = false; // Le joueur est à droite
            }
        }
        else if (distanceInitiale > 0.1f)
        {
            // Retourner à la position initiale si le joueur n'est pas dans la portée
            transform.position = Vector2.MoveTowards(transform.position, positionInitiale.position,
vitesse * Time.deltaTime);
            animator.SetBool("isMoving", true); // Activer l'animation de déplacement
            // Gérer le flip du sprite en fonction de la position initiale
            if (positionInitiale.position.x < transform.position.x)
            {
                spriteRenderer.flipX = true; // La position initiale est à gauche
            }
        }
    }
}
```

```
    }  
    else  
    {  
        spriteRenderer.flipX = false; // La position initiale est à droite  
    }  
}  
else  
{  
    animator.SetBool("isMoving", false); // Désactiver l'animation de déplacement  
}  
}
```

[Unity Learn - Liste et dictionnaires](#)